

LangChain Fundamentals

Introduction

LangChain is a framework for building applications powered by language models, and its core value is not the models themselves but the standardized abstractions it provides for composing model calls, retrieval steps, tool use, and output parsing into a single coherent pipeline. Before adopting the framework, it is worth understanding what problem it actually solves: without it, every RAG pipeline, agent loop, or structured-output workflow tends to be hand-rolled from scratch, with each project reinventing the same prompt-templating, retry, and chaining logic in a slightly different way. LangChain's abstractions exist to make these pieces interchangeable and composable, at the cost of an additional layer of indirection that has to be understood before it can be debugged effectively.

Runnables: The Core Abstraction

Nearly everything in modern LangChain is a Runnable: a standardized interface that exposes the same methods, most importantly `invoke`, `batch`, and `stream`, regardless of what the underlying component actually does. A prompt template, a chat model, an output parser, and a retriever are all Runnables, which means they can all be called the same way and, critically, chained together the same way. This uniformity is what makes composition possible: because every piece exposes the same interface, a developer does not need to learn a different calling convention for a retriever versus a model versus a parser. The trade-off is that this uniformity can obscure what is actually happening underneath a specific `invoke` call, which is why understanding the retrieve-augment-generate loop by hand, as in earlier RAG work, pays off directly once these same steps are wrapped in Runnables.

LCEL: The Pipe Syntax

LangChain Expression Language, or LCEL, is the syntax used to compose Runnables using the pipe operator, written as `|`. A typical chain looks like `prompt | model | output_parser`, which reads left to right as a pipeline: the prompt template is filled in and passed to the model, and the model's raw response is passed to the output parser. This is directly analogous to Unix pipes, where the output of one command becomes the input to the next. The advantage of LCEL over manually writing this logic is that LangChain automatically provides batching, streaming, and async support across the whole chain once each individual piece supports them, without the developer needing to implement that plumbing themselves for every new chain they build.

Prompt Templates

A prompt template is a Runnable that takes a dictionary of variables and produces a formatted prompt, separating the fixed wording of a prompt from the variable content that changes per request. This matters for two practical reasons: it keeps prompt wording centralized and easy to edit without touching application logic elsewhere, and it makes prompts testable, since the same template can be rendered with different inputs and compared. `ChatPromptTemplate` is the variant most commonly used with chat models, and it supports distinct message roles, typically system, human, and AI messages, which map to how most modern chat-based LLM APIs structure a conversation.

Structured Output

Getting a model to reliably return data in a specific shape, such as a JSON object matching a defined schema, is a common requirement once an LLM's output needs to be consumed by other code rather than read by a human. LangChain supports this through structured output binding, commonly implemented with Pydantic models: a developer defines a Pydantic class describing the desired fields and types, binds it to the model, and the framework handles instructing the model to produce output in that shape and parsing the result back into an actual Python object. This removes the need to hand-write brittle regex-based extraction from free-form text, which was a common and fragile pattern before structured output support matured, and it fails more predictably, since a malformed response can be caught as a validation error rather than silently misparsed.

Tool Calling

Tool calling allows a model to request that a specific function be executed, rather than only ever producing text. In practice, a developer defines a set of tools, each with a name, a description, and an expected input schema, and binds them to the model. When the model decides a tool is needed to answer a request, for example a request that requires querying a live database or calling an external API, it returns a structured tool call rather than a plain-text answer. The application code is responsible for actually executing that tool call and returning the result back to the model, which then incorporates it into a final answer. This is the mechanical foundation underneath most agent behavior: an agent is largely a loop that repeatedly checks whether the model has requested a tool call and, if so, executes it and feeds the result back in.

Retrievers as Runnables

Because a retriever is also a Runnable, a vector store built with something like Chroma can be dropped directly into an LCEL chain alongside a prompt and a model, rather than being called with special-case code before the pipeline starts. A typical RAG chain in LCEL might look like retrieving documents, formatting them into a context string, filling a prompt template with that context and the original question, and then passing the result to the model. Conceptually this is identical to the retrieve-augment-generate loop built by hand earlier, and recognizing that identity is the point: LCEL is not introducing a new idea here, it is providing a standard, composable interface around a pattern that was already understood at the code level.

Memory and Conversation State

By default, a single call to a model has no awareness of prior turns in a conversation; each invocation is stateless unless the conversation history is explicitly passed back in.

LangChain provides message history utilities that store prior human and AI messages and re-inject them into the prompt on each new call, effectively simulating memory. This is conceptually simple, the same message list is just appended to and re-sent every time, but it becomes an important practical concern once conversations grow long, since every re-sent message consumes context window budget. Longer-running applications often need a summarization or trimming strategy for older messages, rather than sending the full raw history indefinitely, for the same context-window reasons that limit how many retrieved chunks can be stuffed into a RAG prompt. A common practical pattern is a sliding window combined with periodic summarization: the most recent handful of turns are kept verbatim so recent context stays precise, while older turns are periodically compressed into a running summary that preserves the gist of earlier conversation without paying its full token cost on every subsequent call.

Summary and Practical Checklist

LangChain's value is in standardizing composition, not in introducing fundamentally new capabilities beyond what a hand-rolled pipeline already does. The Runnable interface is what makes every component, from prompts to retrievers to output parsers, callable and chainable in a consistent way, and LCEL's pipe syntax is simply a readable way to express that chaining. Structured output and tool calling both solve the same underlying problem, getting reliable, machine-parseable behavior out of a model that otherwise only produces free text, just applied to two different situations: shaping a final answer versus deciding to call external functionality. The practical test of whether LangChain is being used well is whether a developer can still explain what a given chain does in plain terms without the

framework, since the framework is meant to standardize and simplify composition, not to replace understanding of the underlying mechanics.